ENGINEERING CASE STUDY · MAY 2026

AI SUPPORT COPILOT

RAG-powered document search and grounded answer generation for enterprise support workflows. Citations tracked per response. Auditable by design.

NEXT.JS 14 | FASTAPI | POSTGRESQL 15 | CHROMADB | OPENAI | DOCKER

Hardik Tyagi
AI Product Engineering

THE PROBLEM

Naive GPT Doesn't Scale

NAIVE GPT

- ✕ Hits token limits on large corpora
- ✕ Cost scales with doc size, not query count
- ✕ No source traceability
- ✕ Hallucinates when context is missing
- ✕ No tenant isolation

RAG

- ✓ Top-k chunks only — fixed prompt size
- ✓ Fixed cost per query
- ✓ Claims map to sources
- ✓ Refuses without sufficient context
- ✓ Namespace-isolated per tenant

SYSTEM ARCHITECTURE

Seven Layers, Two Data Paths

Ingestion is async + durable. Retrieval is sync + latency-critical. The two paths share only the vector store and metadata DB.

LAYER	Components	Role
FRONTEND	Next.js · Zustand · TanStack	Chat, upload, citations
API GATEWAY	FastAPI · CORS · tenant extractor	Routing · middleware
INGESTION	PDF parser · chunker · embeddings	Async write path
RETRIEVAL	Embeddings · ChromaDB · threshold	Sync query path
GENERATION	GPT-4.1-mini · prompt · citation repair	Grounded answer
OBSERVABILITY	query_logs · structured logs	Audit · latency · quality
INFRA	Docker Compose · PG 15 · volumes	Deploy · persist

KEY PERFORMANCE METRICS

370ms	24ms	13ms	4.5s	91.7%
UPLOAD HANDOFF	QUERY EMBED	CHROMA SEARCH	GENERATION LATENCY	CITATION COVERAGE

INGESTION PIPELINE

Async Document Processing

HTTP handler returns 202 immediately. Background worker owns the rest. Frontend polls `/status` with exponential backoff.

UPLOAD	→	EXTRACT	→	CHUNK	→	EMBED
202 Accepted		PDF → text		overlap windows		dual write

202 PATTERN

Decouples upload latency from processing. Slow docs never block the HTTP layer.

DUAL WRITE

Chunks go to both PostgreSQL and ChromaDB. Postgres is the source of truth; Chroma is the index.

RETRIEVAL PIPELINE

Semantic Search

Retrieval quality is the ceiling. Wrong chunks guarantee wrong answers — no prompt engineering compensates.

1. **EMBED QUERY**
   Same model used at ingestion. Embedding model mismatch causes silent precision collapse.
2. **CHROMA SEARCH**
   Cosine similarity over tenant namespace. Returns top-k candidates. Latency: **13ms**.
3. **THRESHOLD FILTER**
   Cutoff: `score ≥ 0.55`. Started at 0.65 — lowered after eval showed better recall with no material increase in hallucination rate.
4. **ASSEMBLE CONTEXT**
   Chunks sorted by score, labeled `[Src N]` before reaching the model. Zero raw text without a score gate.

/query/inspect — retrieval-only debug endpoint. Exposes chunk scores, threshold, and per-stage latency without incurring LLM cost. This saved significant debugging time in production.

GENERATION LAYER

Hallucination Defense

L1 · SYSTEM PROMPT

Answer only from provided context. If context is insufficient, say so — do not infer.

L2 · CONTEXT GATE

Only scored, threshold-passing chunks reach the model. Labeled `[Src N]`.

L3 · CITATION REPAIR

If response lacks `[Src N]` markers, a repair pass fires. Uncited answers are not served.

CITATION REPAIR — BEFORE / AFTER

FIRST PASS	AFTER REPAIR
0.0	**0.917**
Citation coverage. Response generated, no [Src N] markers present.	91.7% coverage. 5 sources cited. Cost: 1 extra LLM call, ~4–6s latency.

Citation coverage after repair **91.7%**

Track per tenant over time — citation coverage is a proxy for hallucination rate.

OBSERVABILITY

Structured Audit Layer

Every request gets a UUID at middleware. `request_id` and `tenant_id` thread through the full stack.

STRUCTURED LOG EVENTS

`request_completed`	method, path, status_code, duration_ms, tenant_id
`llm_call_completed`	model, tokens, total_ms
`citation_repair_triggered`	request_id, original_coverage
`ask_completed`	answered, citations_count, total_ms

```sql
CREATE TABLE query_logs (
  id            UUID PRIMARY KEY,
  tenant_id     VARCHAR,
  request_id    UUID,
  query_text    TEXT,
  latency_ms    INT,
  was_answered  BOOLEAN
);
```

Every record keyed by `tenant_id`. Enables per-tenant quality review, latency dashboards, and support audits without cross-tenant data exposure.

MULTI-TENANCY

Namespace Isolation

Tenant identity extracted at middleware. Every ChromaDB search, every DB write, every log entry is scoped to `tenant_id`. No cross-tenant data path exists.

VECTOR ISOLATION

ChromaDB namespace per tenant. A query in `acme-corp` namespace cannot surface chunks from `demo-tenant`.

METADATA ISOLATION

Every Postgres row carries `tenant_id`. All query_logs, document records, and chunk rows are filterable by tenant.

EXAMPLE NAMESPACES

`acme-corp` `demo-tenant` `default` `enterprise-x`

Production P0: Move from client-header tenant extraction to JWT-verified tokens. Current design trusts the `x-tenant-id` header — sufficient for dev but not for multi-tenant production.

FRONTEND ARCHITECTURE

State Separation

Zustand owns UI state. TanStack Query owns server state. Not mixed.

Chat UI	Multi-turn grounded chat, inline `[Src N]` markers, latency footer (5 sources · 13625ms).
Citation Renderer	Parses markers → numbered badges with filename + page tooltip. Footer deduplication at render time.
Upload Interface	PDF drag-and-drop → multipart POST → TanStack polling until `status:ready`. Exponential backoff.
Retrieval Inspector	Dev panel calling `/query/inspect`. Chunk scores, threshold, per-stage latency — no LLM call.

PRODUCTION BUGS ENCOUNTERED

Real Failures, Real Fixes

BUG	ROOT CAUSE	LESSON
No results, no error	Docs ingested to `demo-tenant`; queries hit `default`	Check tenancy first. Silent empties are still bugs.
Every query 404'd	Frontend called `/api/v1/query`; backend served `/query/ask`	Lock the API contract before any frontend work.
Status poll 500	`document_chunks_count` vs `chunk_count` field name mismatch	Validate response models with real serialization paths.
Chroma state in git	Local SQLite committed inside `./chroma-data/`	Keep runtime data out of the repo. Add to `.gitignore` from day one.

Key pattern: `/inspect` exposed retrieval issues in seconds without paying LLM costs. A zero-cost debug path is not optional — it's infrastructure.

ROADMAP

P0 + Quality Improvements

P0 — BEFORE REAL TRAFFIC

- → Celery + Redis — durable, retryable ingestion queue
- → SSE streaming — reduce time to first token
- → JWT auth — tenant from verified token, not header
- → Transactional dual-write — saga or pgvector migration

QUALITY & SCALE

- → Cross-encoder re-ranking (Cohere Rerank)
- → Per-tenant threshold config via eval set
- → Hybrid retrieval — BM25 + semantic via RRF
- → HyDE + query expansion for sparse-query recall

KEY ENGINEERING LEARNINGS

Retrieval quality is the ceiling — optimize it before touching the prompt. Embedding model choice is a commitment — store `model_version` from day one. Structured logging with request IDs makes every debug session trivial. Citation coverage is a proxy for hallucination rate — track it per tenant over time.

ENGINEERING EVIDENCE — PRODUCTION SYSTEM

Chat UI — Inline Citation Grounding

```
// Multi-turn grounded response with source markers
message: "What is the refund policy for enterprise plans?"
response: "Enterprise plans support full refunds within 30 days [Src 1].
  Partial refunds are prorated after 30 days [Src 2][Src 3]."
sources: [policy.pdf p.4, billing-faq.pdf p.2, billing-faq.pdf p.7]
latency: 13625ms   citations: 5 sources
```

Multi-turn conversation with inline [Src N] markers and source attribution footer.

Retrieval Inspector — /query/inspect

```
POST /api/v1/query/inspect
// Zero-cost debug — no LLM call
"retrieval": {
  "query_embedding_ms": 24,
  "search_ms": 13,
  "chunks_retrieved": 8,
  "chunks_passed_threshold": 5,
  "score_threshold_used": 0.55
}
```

Chunk scores, threshold filtering, and per-stage latency — no LLM cost incurred.

Citation Repair — Structured Log Output

```
INFO  citation_repair_triggered  request_id=4f2a...b8  original_coverage=0.0
INFO  llm_call_completed  model=gpt-4.1-mini  total_ms=4821
INFO  ask_completed  answered=true  citations_count=5  coverage=0.917
```

91.7% citation coverage after repair loop. 5 distinct sources. Latency cost: ~4–6s.

Async Ingestion Status — 202 Pattern

```
GET /api/v1/ingest/documents/{id}/status
200 OK
{
  "document_id": "f7ca254-802e-4f7a-bb54-44997b605a2e",
  "status": "ready",
  "chunk_count": 41,
  "updated_at": "2026-05-22T11:45:37.787781"
}
```

202 Accepted with job reference. Frontend polls until status:ready with exponential backoff.